

Secure Document Management System

for Law Enforcement & Legal Case Records

Workflow & Feature Implementation Reference — FastAPI / PostgreSQL / MinIO Architecture

Contents

1. Authentication
2. Basic Hierarchical RBAC
3. Case Management
4. Case-Level Access Assignment
5. PDF Upload
6. SHA-256 Hashing
7. AI-Assisted PDF Extraction
8. PostgreSQL Full-Text Search
9. Search Authorization Filtering
10. Document Versioning
11. Document Approval Workflow
12. Audit Timeline
13. Integrity Verification
14. Failure + Retry Handling

1. Authentication

Establishes who is making a request before anything else in the system runs.

How this project implements it

- Officers log in with email + password via POST /auth/login.
- Passwords are stored as salted hashes, never in plaintext.
- On successful login, FastAPI issues a signed JWT containing the user ID, role, and department.
- Every subsequent API call must carry the token as Authorization: Bearer <JWT>.
- A FastAPI dependency verifies the token signature and expiry on every request before any route logic runs.

Flow

```
Officer  -> POST /auth/login  -> FastAPI
                        |
                    Verify user + password hash
                        |
                    Generate JWT
                        |
                    Return JWT to client
```

Every later request: Authorization: Bearer <JWT> -> Verify signature + expiry -> User identified

2. Basic Hierarchical RBAC

Controls what an authenticated user is allowed to do, based on role, department, and jurisdiction.

How this project implements it

- Each user has a role (Investigating Officer, Senior Officer, Prosecutor, Admin, etc.) and a department (Police, Prosecution, Court, Administration).
- Role and department are kept as separate attributes rather than combined role names, to keep the permission model simple.
- A permission table defines which actions (VIEW, DOWNLOAD, EDIT, APPROVE, DELETE) each role may perform.
- Authorization decisions never trust role data sent by the client; the server always re-checks against the stored user record.
- A single reusable authorization dependency/service is called from every route, instead of duplicating permission checks per endpoint.

Flow

```
Authenticated user
|
Load role + department (server-side, trusted)
|
Is this action permitted for this role?
|
YES / NO  ----> proceed to case/document level checks
```

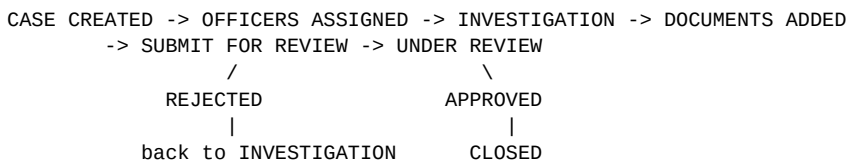
3. Case Management

Groups every document, officer, and workflow step under a single Case, which becomes the central object the rest of the system revolves around.

How this project implements it

- A case is created with a case number, jurisdiction, and an owning officer.
- Case status moves through a defined lifecycle: CREATED -> INVESTIGATION -> UNDER_REVIEW -> APPROVED / REJECTED -> CLOSED.
- A case dashboard shows assigned officers, linked documents, versions, and the audit/case timeline in one place.
- Once a case is CLOSED, new documents, new versions, and edits are blocked; only viewing and audit access remain available.
- The case timeline reuses the same underlying audit events that power document-level audit logs, rather than a separate logging system.

Flow



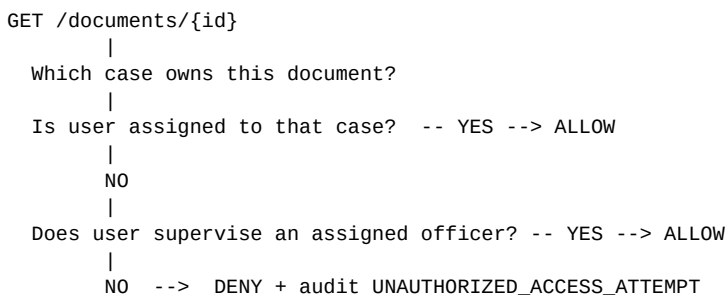
4. Case-Level Access Assignment

Restricts document access to only the officers who are actually assigned to that case, instead of granting every officer in a department blanket access.

How this project implements it

- Each case stores its assigned officers with an assignment type (Investigator, Supervisor).
- A supervising officer's authority extends to the cases of the officers they supervise.
- When a request comes in for a document, the system checks: is this user assigned to the owning case, or do they supervise someone who is?
- If neither condition is met, the request is denied and the attempt is written to the audit log as an unauthorized access attempt.
- This turns case assignment into a first-class authorization input, not just descriptive metadata.

Flow



5. PDF Upload

Accepts a document from the officer and stores it safely without making the user wait for downstream processing.

How this project implements it

- The React client sends the file to POST /upload as a multipart UploadFile.
- FastAPI validates the file, stores the original PDF in MinIO object storage, and creates a PostgreSQL record with status = PROCESSING.
- The heavier work (OCR + AI extraction) is handed to a FastAPI BackgroundTask so the API can respond immediately.
- The response returned to the client is just the new document ID and its current status, e.g. { document_id: DOC-8891, status: PROCESSING }.

Flow

```
React Client -> POST /upload -> FastAPI
                        |
                        Validate file + store in MinIO
                        |
                        Create PostgreSQL record (status = PROCESSING)
                        |
                        Hand off to BackgroundTasks (OCR + AI)
                        |
                        Return { document_id, status: PROCESSING }
```

6. SHA-256 Hashing

Gives every stored file version a cryptographic fingerprint, which is the foundation for tamper detection later.

How this project implements it

- A SHA-256 hash is calculated at upload time, before the file is written to MinIO.
- The hash is stored alongside the document/version metadata in PostgreSQL, tied to that specific version.
- Every new version created for a document gets its own independently calculated hash; existing hashes are never recalculated or overwritten.
- This hash is later re-used both for integrity verification and for the audit trail (see sections 12 and 13).

Flow

```
Uploaded PDF bytes
      |
      Calculate SHA-256
      |
      Store hash with document_version record
      |
      (hash reused later for Integrity Verification)
```

7. AI-Assisted PDF Extraction

Converts an unstructured scanned document into structured, searchable case metadata automatically.

How this project implements it

- The stored PDF is converted to page images with pdf2image, then run through pytesseract OCR to produce raw text.
- The raw OCR text is sent to an LLM (Gemini/OpenAI) with a strict extraction prompt that asks only for defined fields: fir_number, incident_date, police_station, complainant, accused, ipc_sections, document_type.
- The LLM is instructed to return only valid JSON, which is parsed and validated before being written to PostgreSQL.
- Once metadata and text are stored and indexed, the document status is updated from PROCESSING to READY.

Flow

```
PDF -> pdf2image -> Page Images -> pytesseract -> Raw OCR Text
                                                    |
                                                    LLM (Gemini/OpenAI) + extraction prompt
                                                    |
                                                    Structured JSON metadata
                                                    |
                                                    Parsed, validated, stored in PostgreSQL
                                                    |
                                                    status = READY
```

8. PostgreSQL Full-Text Search

Lets officers search across OCR-extracted document text quickly, without a separate search engine.

How this project implements it

- Raw OCR text for each document is converted into a PostgreSQL tsvector and stored as a search_vector column.
- A GIN index is created on search_vector for fast lookups: CREATE INDEX idx_gin_search ON documents USING gin(search_vector).
- User queries are parsed with websearch_to_tsquery() rather than raw to_tsquery(), so normal typed queries behave predictably.
- Matches are ranked with ts_rank() and shown with a contextual snippet generated by ts_headline().

Flow

```
Search Query -> FastAPI /search -> PostgreSQL
                                                    |
websearch_to_tsquery() @@ search_vector (GIN index)
                                                    |
ts_rank() (relevance) + ts_headline() (snippet)
                                                    |
Ranked Search Results
```

9. Search Authorization Filtering

Ensures full-text search never surfaces a document the requesting officer isn't authorized to see.

How this project implements it

- A raw `SELECT ... WHERE search_vector @@` query is treated as unsafe on its own, since it ignores case/document permissions.
- Before results are returned, the officer's accessible case/document set is determined the same way as for direct document access (role + case assignment + explicit permission).
- The full-text match is intersected with that accessible set, so only authorized matches are ever returned to the client.
- In the worked example, 20 raw text matches are filtered down to 7 documents the officer is actually permitted to see.

Flow

```
Search: "blue pulsar"
|
PostgreSQL full-text match ----> 20 raw matches
|
Access-control filter (role + case assignment + permission)
|
Authorized matches only ----> 7 results returned
```

10. Document Versioning

Preserves every prior state of a document instead of overwriting it, so history can never be silently lost.

How this project implements it

- An existing version is never modified. Any edit creates a brand new `document_versions` record with its own file, hash, creator, and timestamp.
- MinIO stores each version as a separate object (`v1.pdf`, `v2.pdf`, ...); PostgreSQL tracks `documents.current_version` and the full `document_versions` history.
- Viewing an older version never changes which version is current; current vs. historical are always kept distinct.
- Restoring an old version creates a new version entry (e.g. `V5`, restored from `V1`) rather than rewinding `current_version`, so the chronological history stays intact.
- Once a version is `APPROVED`, it is locked; any further change must go through a new version and a new review cycle.

Flow

```
Version 1 (hash ABC123)
| officer modifies content
v
Version 2 (hash XYZ789)  <- Version 1 stays untouched, still retrievable

documents.current_version = 2
document_versions: [ v1 -> ABC123, v2 -> XYZ789 ]
```

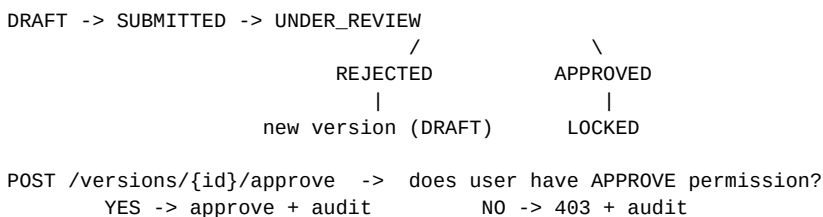
11. Document Approval Workflow

Adds a controlled review/sign-off step before a document version becomes authoritative, mirroring how case documents move through real review chains.

How this project implements it

- Each version carries its own status: DRAFT -> SUBMITTED -> UNDER_REVIEW -> APPROVED (or REJECTED, sent back for modification).
- The APPROVE action is permission-gated: an Investigating Officer cannot approve their own submitted version; only a role with APPROVE rights (e.g. Senior Officer) can.
- The backend enforces this at the API level, not just by hiding the Approve button in the UI.
- A rejected version requires a new version to be created for the next review cycle rather than editing the rejected one in place.

Flow



12. Audit Timeline

Keeps a permanent, append-only record of every security-relevant action, answering who did what, to which document, when, and with what result.

How this project implements it

- A single centralized `create_audit_log(user, action, document, case, result, details)` function is called from every sensitive operation (login, upload, view, download, update, approve, deny, integrity check, unauthorized attempt).
- Both successful actions and denied attempts are logged, so repeated unauthorized access attempts are visible, not just successful ones.
- Application-level permissions make audit records effectively append-only: normal users can create logs indirectly (via actions) but cannot update or delete them.
- Each audit record can additionally store the hash of the previous record (a hash chain), so if any past record is altered, the chain breaks and tampering is detectable.
- The same audit events power two views: a per-document audit log, and a per-case timeline (e.g. case created -> officer assigned -> FIR uploaded -> submitted for review -> approved).

Flow

Audit 1 (hash H1) -> Audit 2 (prev H1, hash H2) -> Audit 3 (prev H2, hash H3)

If Audit 2 is altered: H2 changes -> Audit 3.previous_hash != H2 -> tampering detected

Example entries:

```
18:42 Officer A uploaded Version 1      (SUCCESS)
19:02 Officer B attempted case access   (DENIED)
19:30 Integrity verification             (FAILED)
```

13. Integrity Verification

Detects whether a stored file's content has been altered since it was hashed, independent of who accessed it.

How this project implements it

- Verification recalculates the SHA-256 hash of the file currently stored in MinIO for a given version.
- The recalculated hash is compared against the hash recorded for that version at creation time.
- A mismatch is written to the audit trail as an INTEGRITY_CHECK / FAILED event, with both the expected and actual hash values recorded.
- A failed check flags the document for investigation rather than silently allowing continued access, giving a clear, demonstrable chain: hashing -> versioning -> audit -> tamper detection.

Flow

```
Stored file (this version)
|
Recalculate SHA-256
|
Compare to hash recorded at creation
|
MATCH -> OK                      MISMATCH -> audit INTEGRITY_CHECK: FAILED
                                   -> document flagged for investigation
```

14. Failure + Retry Handling

Keeps the system in a known, recoverable state when background processing (OCR / AI extraction) fails partway through, instead of leaving a document stuck or silently incomplete.

How this project implements it

- If OCR or the LLM extraction step throws an error, the background task catches it and sets the document status to PROCESSING_FAILED instead of leaving it at PROCESSING indefinitely.
- The failure reason (e.g. OCR error, LLM timeout, invalid JSON returned) is recorded on the document record and as an audit event.
- A retry endpoint re-queues only the failed stage (OCR or extraction) as a new BackgroundTask, without requiring the officer to re-upload the original file, since the original PDF in MinIO is untouched.
- The document notification (see below) alerts the uploading officer when processing fails, so it does not go unnoticed.
- A capped retry count prevents an unrecoverable file (e.g. corrupted scan) from retrying indefinitely; after the limit, it is left in PROCESSING_FAILED for manual review.

Flow

```
PROCESSING -> OCR/LLM step throws error -> PROCESSING_FAILED
                                           |
                                           audit + notify uploading officer
                                           |
POST /documents/{id}/retry -> re-queue failed stage
                                           |
SUCCESS -> READY                      FAILURE (n times) -> manual review
```